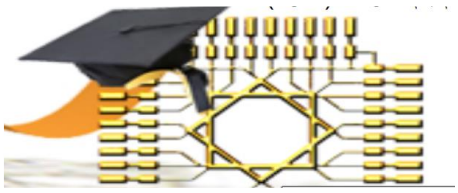
الموضوع: التسليم الأولى

اعداد: أنس نوفل

المشرف: ما.محمد بشار دسوقي

المقرر: الحوسبة المتوازية



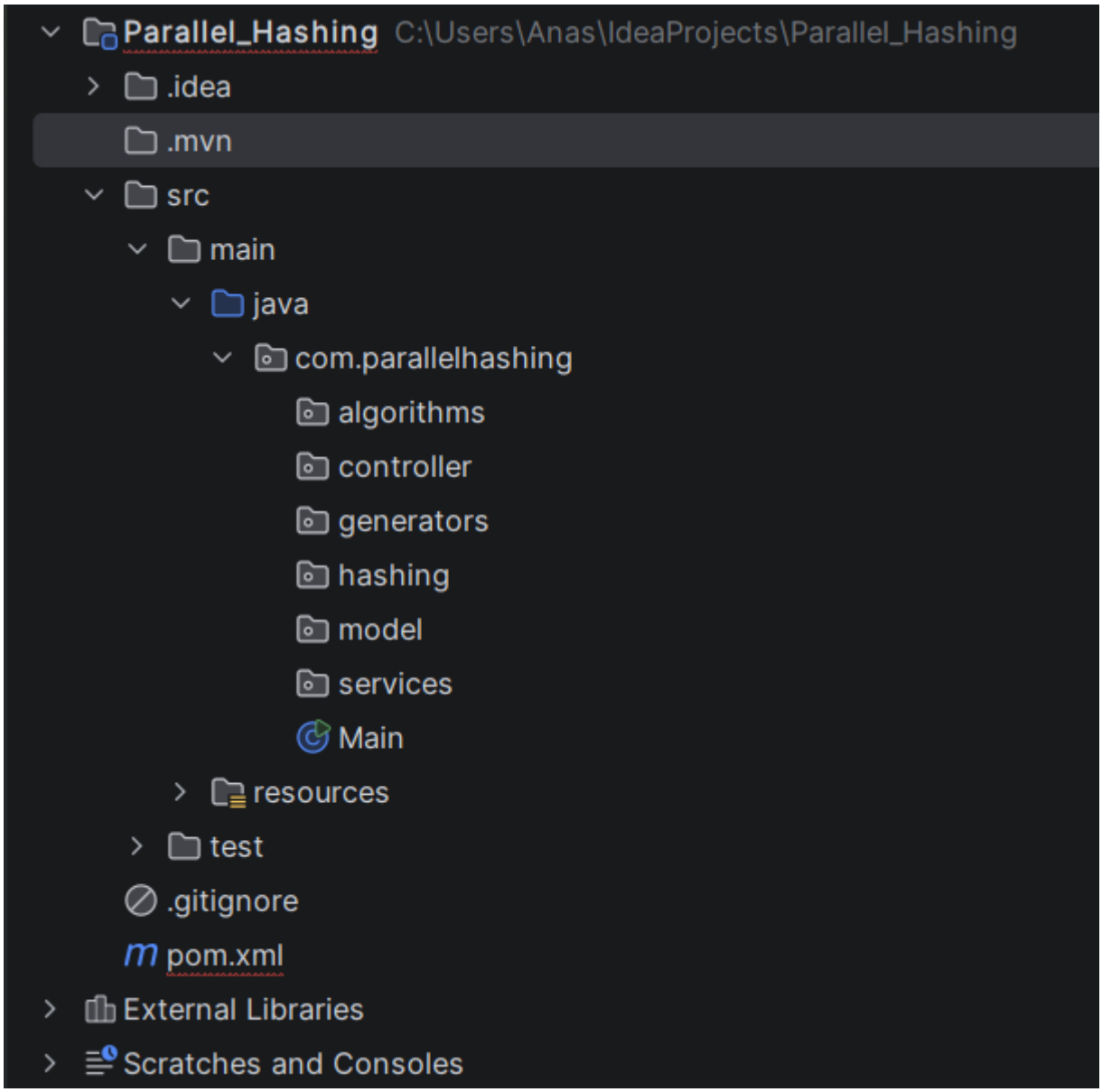
المعهد العالي للعلوم التطبيقية والتكنلوجيا

Parallel_Hashing

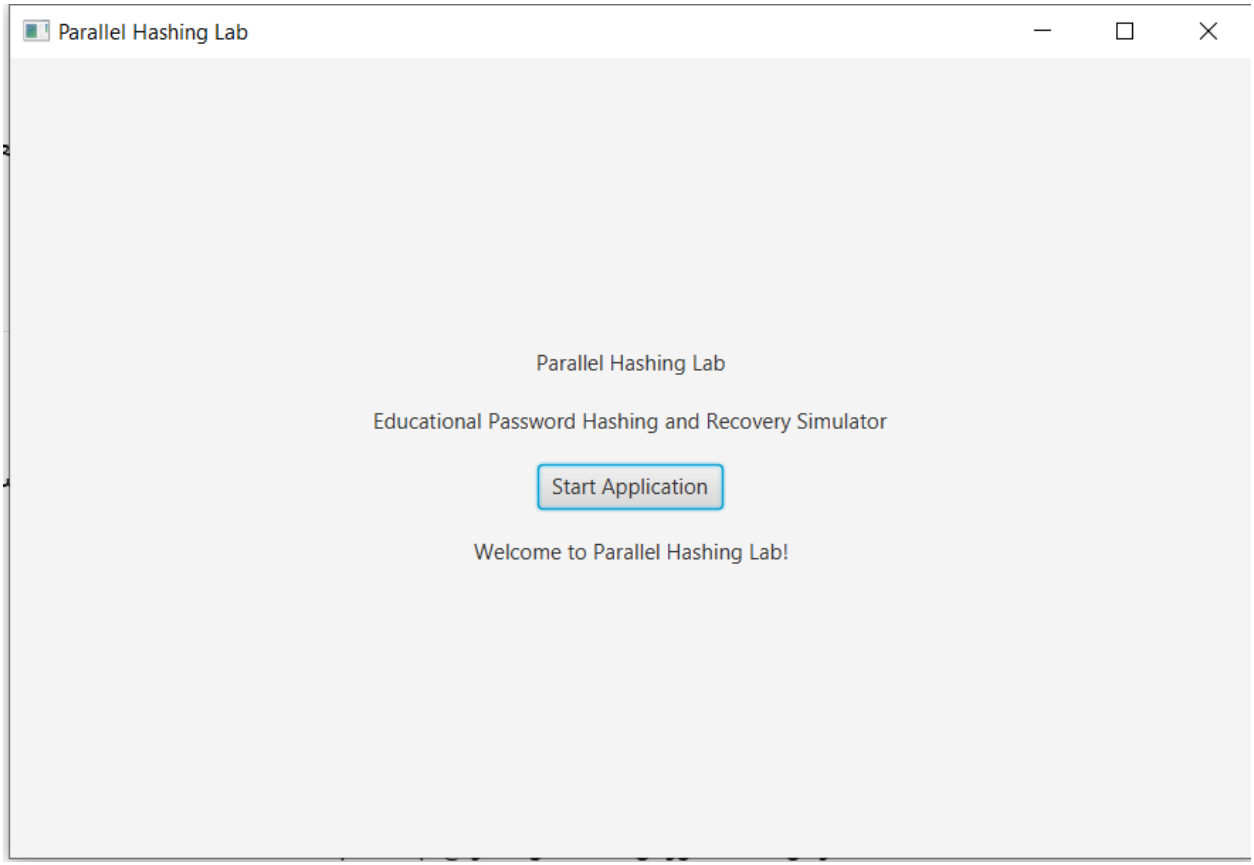
قمت ببناء تطبيق يقوم باخذ hash من المستخدم ويقوم بمحاولة ايجاد الكلمة التي تصلح لتكون كلمة السر من خلال ايجاد كل الاحتمالات الممكنة اي هذا يشبه هجوم Brute Force

فضلاً و ليس أمراً حمل التطبيق المرفق مع المشروع الموجود على github

يبين هذا الشكل الهيكل العام للمشروع



يجب في البداية ان نقوم بانشاء الواجهة وهي تظهر هكذا



طبعاً هذه واجهة أولية وسنقوم ببناء كامل المشروع في الخطوت القادمة حتى نحصل على النموذج النهائي.

لقد استخدمت JavaFX من أجل تحقيق هذه الواجهة.

الان سأقوم بانشاء ملف mainController.java داخل Controller ووظيفته هي ان يكون وسيطاً بين الواجهة والمكونات البرمجية، مثلاً عندما أقوم بضغط start يستقبل handleStart يستقبل المدخلات ويتحقق من صحتها

تالياً لقد قمنا بانشاء ملف HashUtils.java داخل hashing والذي يحتوي على الوظائف الثلاثة التالية

الوظيفة الاولى: sha256

تستقبل نصاً من نمط string وتحوله الى مصفوفة من البايتات، ثم تحسب بصمته باستخدام SHA-256، وأخيراً تحول الناتج الى سلسلة من الاحرف الست عشرية

الوظيفة الثانية: isValidSha256Hash

هذه الوظيفة تقوم بالتحقق من ان عدد المحارف المدخل هو 64 محرفاً، وكلها ارقام أو احرف من A الى F

نختار 64 لأن sha256 ينتج 256 بت، وكل محرف Hexadecimal يمثل 4 بت:

256/4=64

الوظيفة الثالثة: matches

تأخذ كلمة والhash المستهدف، ثم تقوم بحساب sha256 للكلمة المختارة وتقارنها بالـhash المستهدف

بعدها قمت بانشاء ملف SearchResult.java الموجود في model

لا نريد أن نعيد كلمة فقط، بل نريد أن نعيد معلومات أكثر

Found : هل تم ايجاد تطابق

matchedCandidate: ما هي الكلمات التي تطابقت

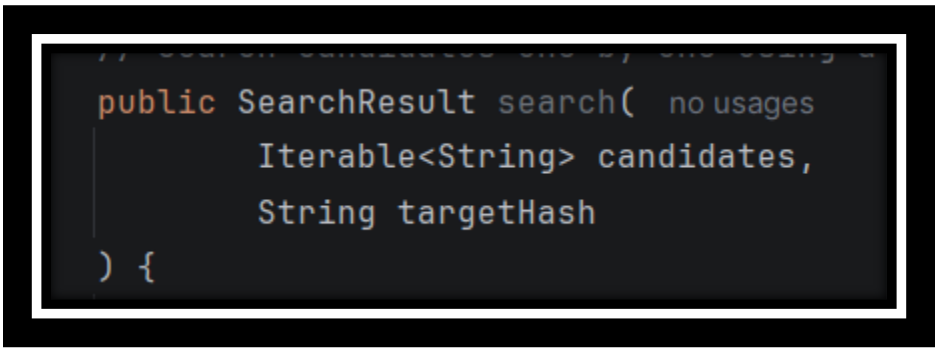
hashesChecked: كم عدد البصمات التي تم حسابها

elapsedNanos: كم استغرقت العملية بالنانوثانية

كما واستخدمنا record لأنه مناسب لتخزين بيانات النتيجة بشكل مختصر وغير قابل لتعديل بعد انشائه.

بعد ذلك قمنا بانشاء ملف SequentialEngine.java المتواجد في algorithms

استقبال المدخلات:



كما نلاحظ في هذا القسم من الكود لدينا مدخلان candidates: وهي مجموعة الكلمات التي نريد اختبارها
وtargetHash: البصمة التي نبحث عن كلمة تطابقها

واستخدمنا Iterable حتى لا نربط المحرك بنوع قائمة واحد، يمكنه التعامل مع اي مجموعة يمكن المرور على عناصرها بالتتابع.

قياس الزمن:

```
long startTime = System.nanoTime();
```

نسجل وقت البداية بالنانوثانية، ثم بعد انتهاء العملية نطرح وقت البداية من وقت النهاية لنحصل على المدة التي اساغرقتها عملية البحث

```
long elapsedTime = System.nanoTime() - startTime;
```

لقد استخدمنا System.nanoTime() لأننا نقيس مدة عملية قصيرة. وnanoTime مصمم لقياس الفترات الزمنية القصيرة بدقة.

المرور على الكلمات:

```
for (String candidate : candidates)
```

هذه الحلقة تمر على كل كلمة بالتتابع.

اعادة النتيجة:

اذا وجدنا تطابقاً نعيد الكلمة وعدد المحاولات والزمن. واذا انتهت الكلمة دون تطابق، نعيد found=false و matchedCandidate = null

الان سننتقل الى بناء ParallelEngine

كيف سنقسم المهام:

سنستخدم معادلتين لتخصيص الجزء المناسب لكل thread

لنفترض أن N: عدد العناصر الكلي، T: عدد threads، i: رقم Thread يبدأ من الصفر فيكون لدينا

$$Start_i = \frac{i \times N}{T}$$

$$End_i = \frac{(i+1) \times N}{T}$$

بعد هذه المحاكمة قمت بانشاء ملف ParallelEngine.java

أولاً: ExecutorService

```
ExecutorService executor =
        Executors.newFixedThreadPool(threadCount);
```

بدلاً من انشاء Thread في كل مرة قمنا بانشاء threadPool : وهي مجموعة من Threads جاهزة لاستقبال المهام وتنفيذها. عندما نرسل مهمة باستخدام submit يتولى أحد threads المتاحين تنفيذها. كما تساعد threadPool ايضاً في تنظيم انشاء threads وادارتها، واعادة استخدام threads بدلاً من انشاء thread جديد لكل مهمة، وتحديد عدد threads التي تعمل في الوقت نفسه

ثانياً: future

```
Future<?> future = executor.submit(() -> {

    // Process only the assigned range
    for (int j = start; j < end; j++) {

        // Stop if another worker has already found a match
        if (found.get()) {
            break;
        }

        String candidate = Objects.requireNonNull(
                candidates.get(j),
                message: "Candidate cannot be null"
        );

        hashesChecked.incrementAndGet();

        if (HashUtils.matches(candidate, targetHash)) {

            // Only one worker can publish the result
            if (matchedCandidate.compareAndSet( expectedValue: null, candidate)) {
                found.set(true);
            }

            break;
        }
    }
});
```

عندما نرسل مهمة الى threadPool، نحصل على كائن Future، يمكننا من خلاله متبعة المهمة وانتظار انتهائها، كما ونستطيع إلغاءها عند الحاجة.

ثالثاً: AtomicBoolean

```
// Shared state between worker threads
AtomicBoolean found = new AtomicBoolean( initialValue: false);
```

بما أنه لدينا عدة threads تعمل مع بعضها في الوقت نفسه فعند ايجاد الكلمة من قبل thread معين يكون يجب ان نعلم باقي threads بانه قد انتها البحث، وهنا يأتي دور AtomicBoolean

عند عثور أحد threads يغير قيمته الى true وعندها تقرأ باقي threads هذه القيمة وتتوقف

رابعاً: AtomicReference

```
AtomicReference<String> matchedCandidate = new AtomicReference<>( initialValue: null);
```

نستخدمها لتخزين الكلمة التي تم العثور عليها بطريقة آمنة عند مشاركة البيانات بين threads

```
// Only one worker can publish the result
if (matchedCandidate.compareAndSet( expectedValue: null, candidate)) {
```

الكود الموجود فوق الكتابة مباشرة يعني ضع الكلمة الجديدة فقط اذا كانت القيمة الحالية لا تزال null.

كما انه لا يمكن لأي threads التدخل في منتصف عملية المقارنة والتحديث، وبالتالي هذا يمنع اكثر من thread من نشر نتيجة مختلفة باعتبارها النتيجة المرجوة.

خامساً: AtomicLong

```
AtomicLong hashesChecked = new AtomicLong( initialValue: 0);
```

نحتاج الى عد عدد hashs التي تم فحصها، لو قمنا باستخدام long فقد تقوم threads متعددة بزيادة قيمته في الوقت نفسه، وتزيد بعض الزيادات بسبب Race Condition.لذلك استخدمنا hashesChecked.incrementAndGet

وبالتالي نحصل على زيادة آمنة

ملاحظة: عدد hashs في التنفيذ المتوازي قد يختلف عن التسلسلي عند العثور على تطابق. لأن عدة threads قد تكون بدأت فحص عناصرها قبل ان تلاحظ ان thread آخر قد وجد الحل.

مناقشة حدوث Deadlock:

في هذا التطبيق لا نستخدم synchronized أو اقفالاً يدوية متداخلة، ولا ننتظر مهمة أخرى أثناء الاحتفاظ بقفل. وهذا من أحد الاسباب الشائعة للـDeadlock

يجب التنويه الى انه لا يعني هذا الكلام أن كل برنامج يستخدم thread pool محصن تلقائياً من Deadlocks، يجب دائماً تصميم علاقات الانتظار بعناية.

الان وصلت الى تنجيز هذا الكود SearchService.java الذي يتواجد في Service

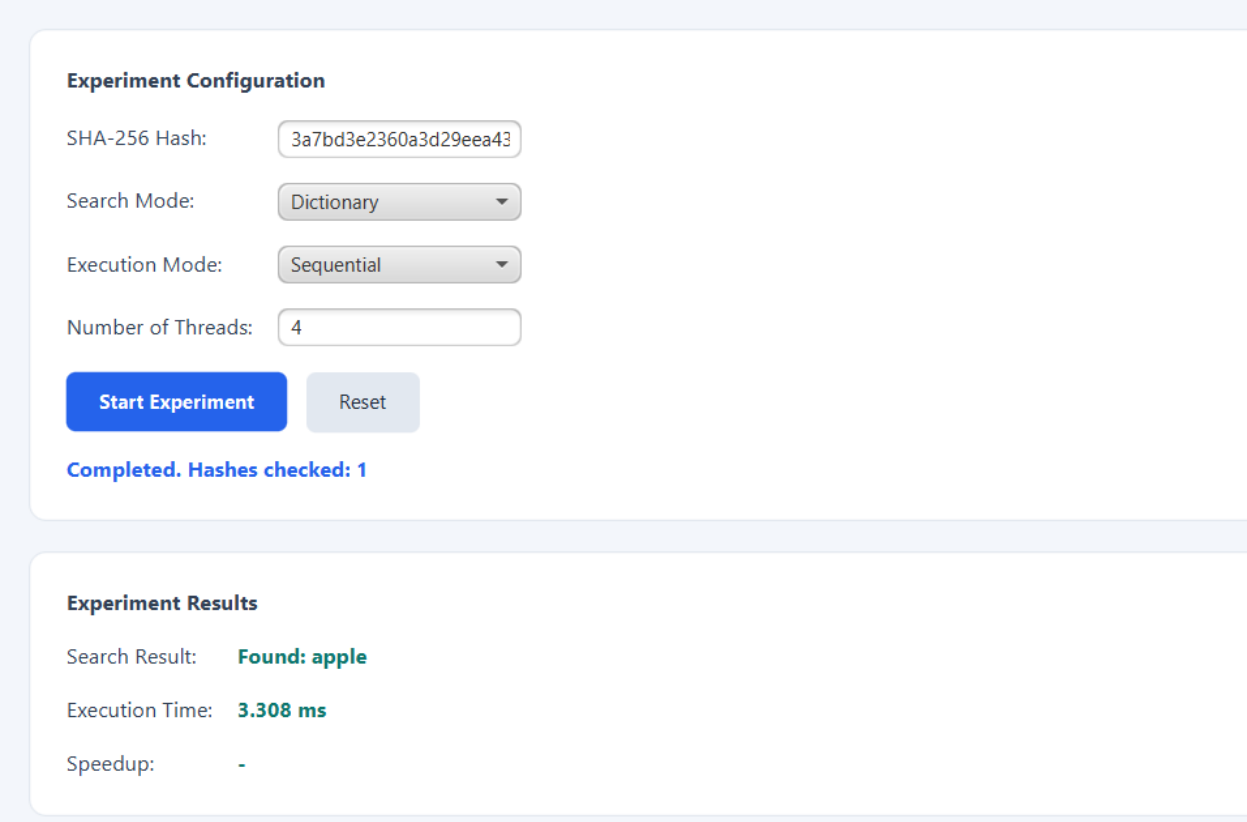
دور هذا الكود هو بدلاً من أن تتعامل الواجهة مباشرة مع SequentialEngine وParallelEngine ستتعامل مع SearchService, وهو بدوره يختار المحرك المناسب

بعد انشاء الكود السابق ننقل الان الى انشاء هذا الكود DictionaryGenerator.java الذي قمت بوضعه في generators

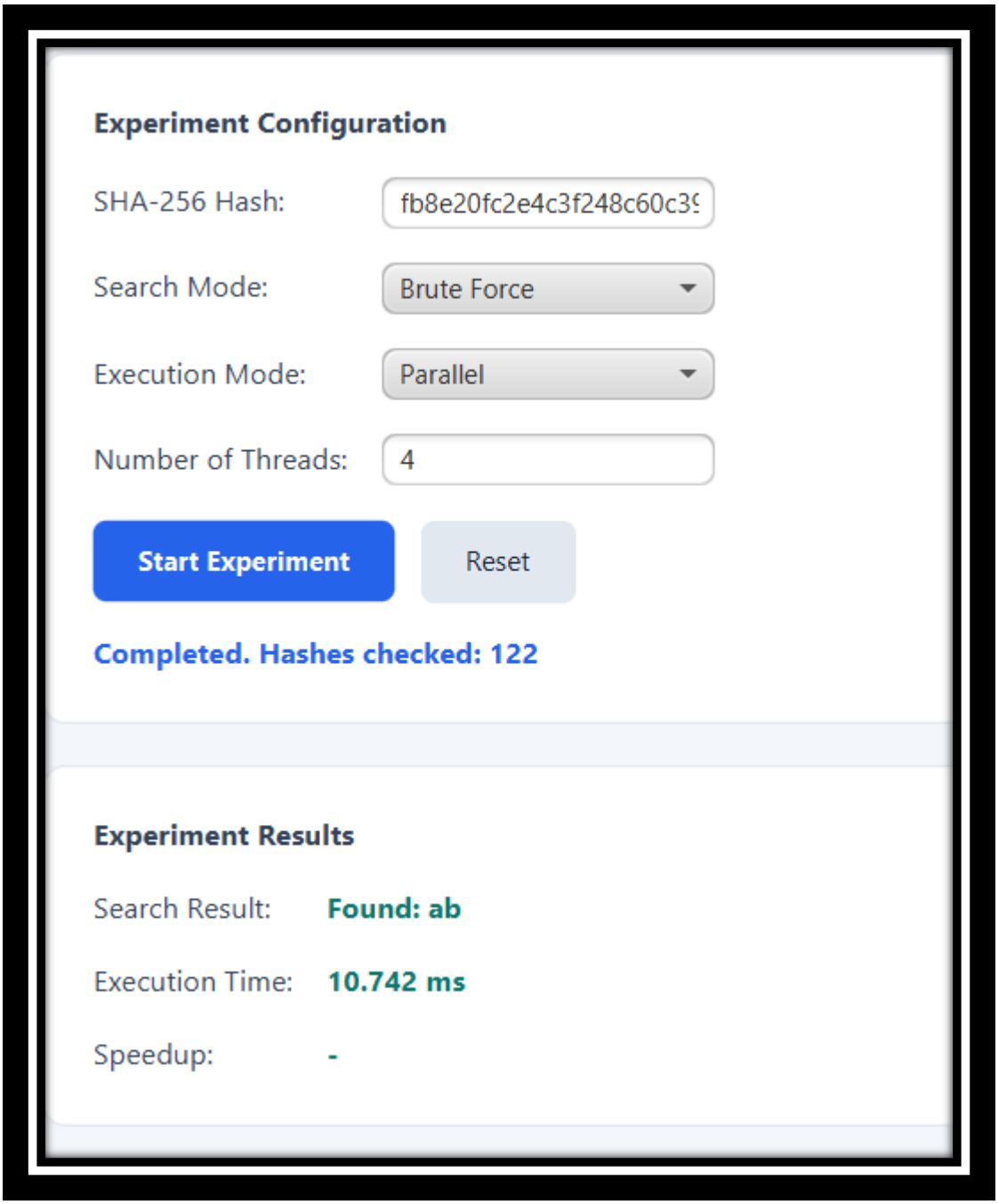
يقوم بهذا الكود بمقارنة الـhash الذي ادخله المستخدم مع مجموعة من hashs التي تم حسابها للكلمات الموجودة في ملف txt

---

اختبارات:

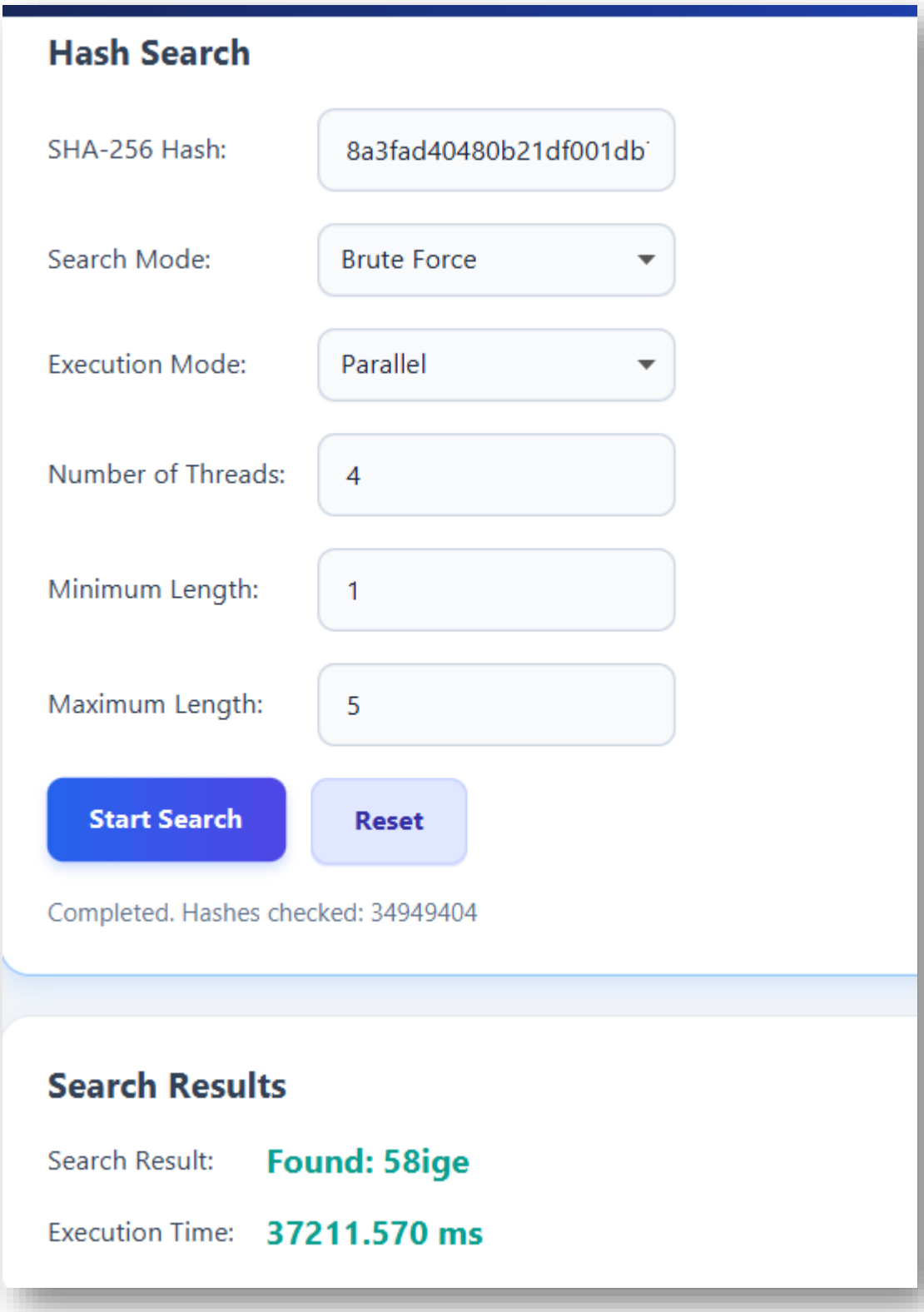


قمت في البداية بحساب hash لكلمة apple وبعدها ادخلت هذا الـhash فحصلت على تطابق من الملف الذي قمت بوضع الكلمات فيه مع اختيار النمط sequential، والنمط قاموس.

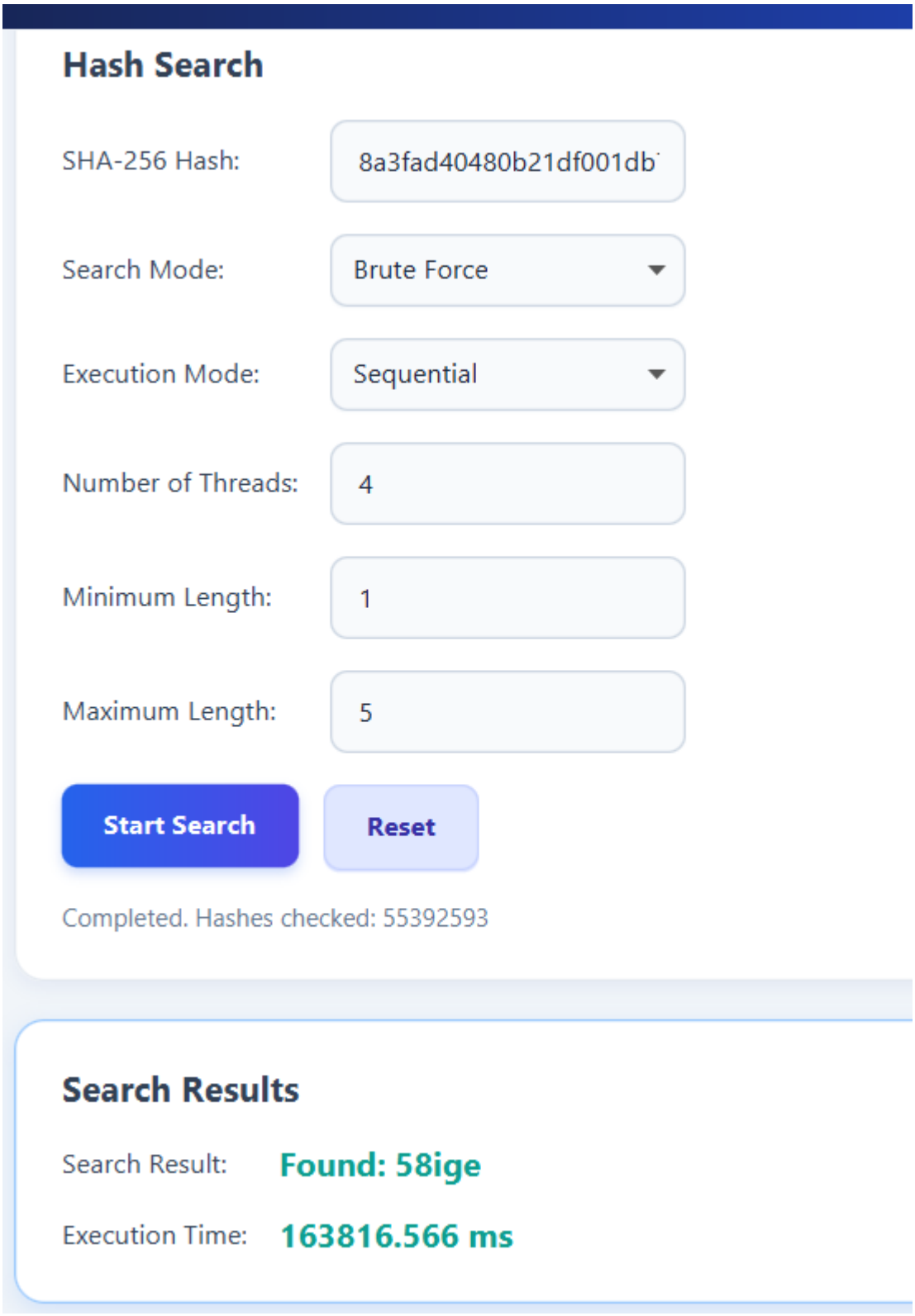


وهذا الاخبار الذي قمت به لايجاد كلمة ab وذلك بعد ادخال hash لها

بعد ذلك قمت بتعديل التطبيق ليصبح بهذا الشكل المبين هنا

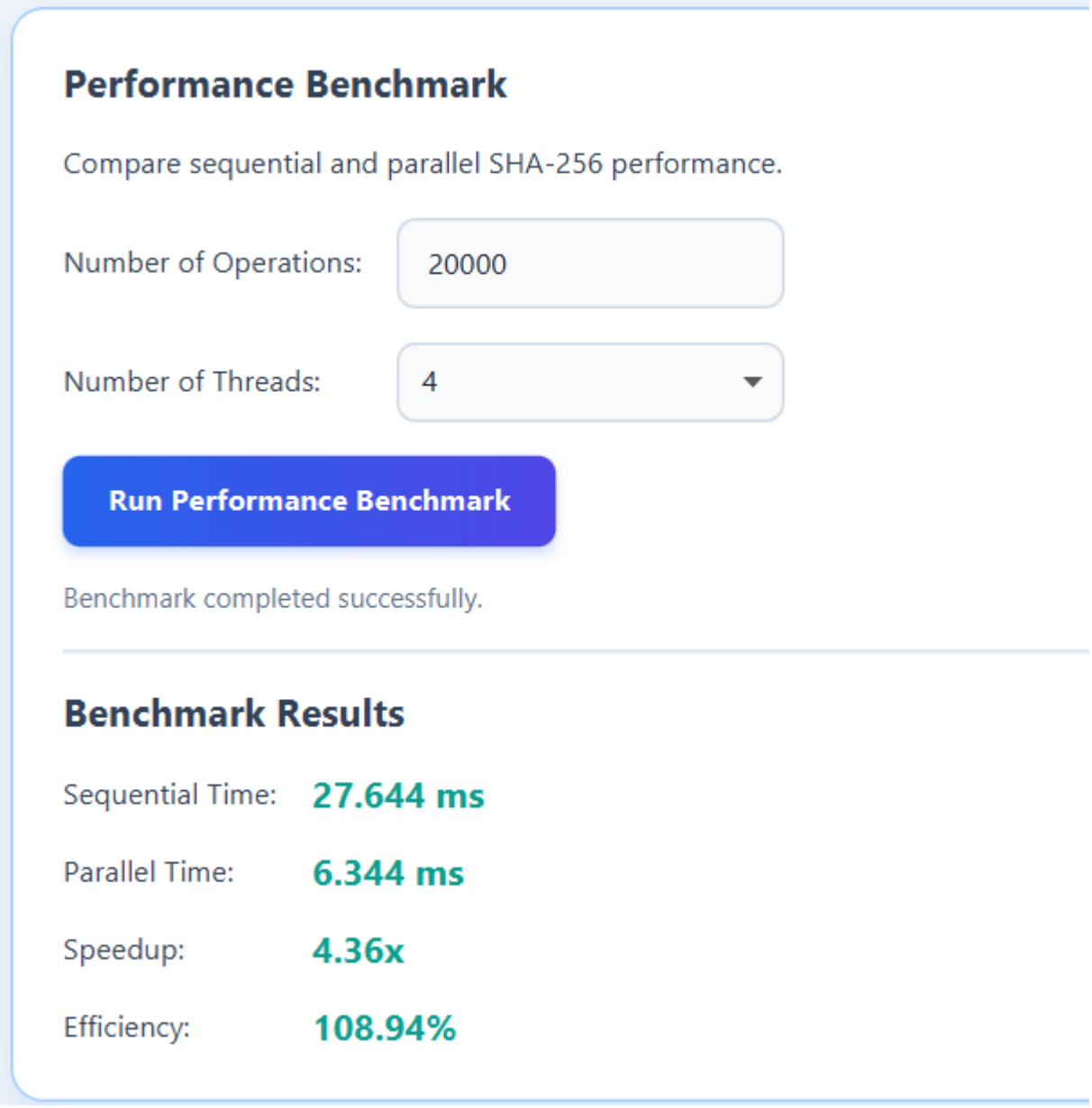

Hash Search
SHA-256 Hash:
8a3fad40480b21df001db
Search Mode:
Brute Force
Execution Mode:
Parallel
Number of Threads:
4
Minimum Length:
1
Maximum Length:
5
Start Search
Reset
Completed. Hashes checked: 34949404
Search Results
Search Result:
Found: 58ige
Execution Time:
37211.570 ms

يقوم المستخدم بادخال hash ونختار نمط Brute Force ونختار parallel أو Sequential ثم نختار عدد threads في حال اخترنا Sequentail ونختار أقل طول متوقع للكلمة وأطول طول متوقع.

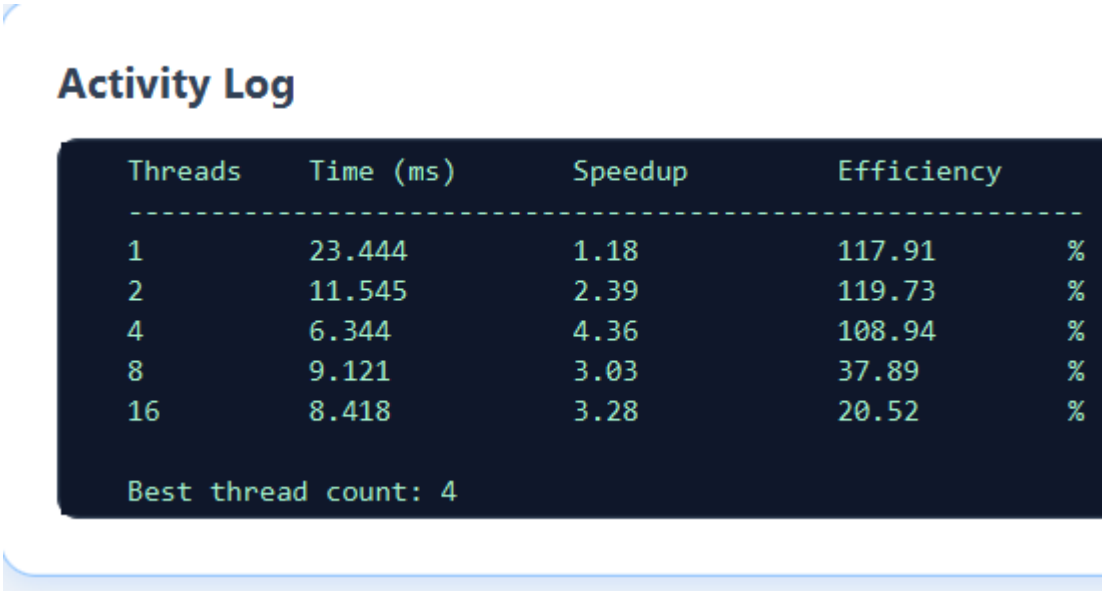


نلاحظ ان اختيار parallel كان افضل من Sequential عند زيادة عدد العمليات، لقد حصلنا على تسريع قدره 4.4

أضفت اضافة اخرى لتطبيق يقوم المستخدم بادخال عدد الاحتمالات المتوقعة بدل ادخال hash ويقوم التطبيق بحساب زمن التنفيذ للعملية من اجل thread أو 2 أو 4 أو 8 أو 16 ثم يعطينا الخرج على شكل جدول ويوضح اي الطرق هس الأفضل يمكن ان يختار المستخدم حتى 5000000 احتمال، توضح الصورة التالية اختبار لهذا الجزء



وهذا الجدول الذي يعرض كل القيم وأيها الافضل



ان اسرع الطرق هو التشغيل على 4 Threads وصل التسريع الى 4.36

ملاحظة: لتحويل الكلمة التي أريد ايجادها باستخدام التطبيق استخدمت هذا الكود HashDemo.java والموجود في Parallelhashing نقوم بوضع الكلمة في string word ونقوم بتشغيل الكود لنحصل على hash للكلمة التي ادخلناها وبعدها ندخل hash في التطبيق.

عند تنفيذ الخوارزمية باستخدام نمط Parallel مع thread واحد، لاحظنا أن قيمة speedup بلغت 1.18 مقارنة بالتنفيذ Sequential. لا تعني هذه النتيجة وجود تسريع ناتج عن التوازي الفعلي. لأن التنفيذ باستخدام thread واحد لا يسمح بمعالجة عدة مهام في الوقت نفسه. ويمكن تفسير هذا الفرق باختلاف آليات إدارة المهام، وتأثير JVM وJIT Compiler، إضافة الى الارتياب في قياسات زمن التنفيذ.

Efficiency: هي مقياس يوضح مدى الاستفادة من عدد الـthreads المستخدمة في تنفيذ مهمة بالتوازي، ومدى فعالية توزيع العمل بينها. تحسب بقسمة قيمة Speedup على عدد threads، وفق المعادلة التالية:

$$Efficiency = \frac{Speedup}{Number\ of\ Threads}$$